

PIL

Technical Architecture Report

V1 — Core ML Pipeline & API Infrastructure

March 2026 | Engineering Confidential

Component	Technology	Version	Purpose
API Proxy	FastAPI	0.111	Async HTTP proxy, provider adapters
Intent Classifier	distilBERT + ONNX	1.3	Prompt classification < 20ms
Token Counter	tiktoken / Anthropic SDK	0.7 / 0.25	Exact token counts per provider
PII Scrubber	Microsoft Presidio	2.2	Entity detection + redaction
Summariser	BART-large-cnn	HF 4.39	Conversation history compression
Vector Store	Qdrant	1.9	Semantic cache, < 5ms query
Embeddings	all-MiniLM-L6-v2	HF 4.39	384-dim semantic similarity
Queue	Celery + Redis	5.3 / 7.2	Async jobs, cache warming
Observability	Prometheus + Grafana	2.51 / 10.4	Metrics, alerting, dashboard
Container	Docker + Kubernetes	26 / 1.29	Cloud-agnostic deployment

1. System Overview

PIL is an API proxy that intercepts HTTP requests destined for LLM provider endpoints, runs a multi-stage ML pipeline to optimise the prompt, and forwards the optimised request to the provider. From the client's perspective it is a drop-in endpoint replacement — the same request shape, the same response shape, with additional informational headers on the response.

Core Design Principle

PIL adds zero new concepts to the developer's workflow. Change one URL. Get better results at lower cost.
Base URL change: `https://api.openai.com` → `https://api.pil.dev/openai`
All other request parameters remain identical. PIL adapters handle provider-specific formatting internally.

1.1 Request Lifecycle

- Client sends POST `/v1/chat/completions` (or equivalent) to PIL proxy
- AuthN middleware validates PIL API key; rate limiting applied
- Input validation: size check (max 128k chars), encoding verification
- PII scrubber runs — entities and secrets replaced with typed placeholders
- Semantic cache lookup — cosine similarity check against Qdrant; cache hit returns immediately
- Intent classifier runs — ONNX inference, classifies task type + language + framework
- Token budget engine scores prompt clauses and compresses to budget
- Structured prompt builder assembles final prompt using intent-matched template
- Async: audit log entry queued to Celery worker; conversation summariser updated
- Optimised prompt forwarded to provider with original auth headers
- Provider response returned to client with X-PIL-* headers appended

1.2 PIL Response Headers

Header	Type	Description
X-PIL-Tokens-In	integer	Token count of original prompt
X-PIL-Tokens-Out	integer	Token count of optimised prompt
X-PIL-Tokens-Saved	integer	Tokens removed by PIL
X-PIL-Compression-Ratio	float	Optimised / original token ratio
X-PIL-Intent	string	Classified intent (e.g. debug:python:fastapi)
X-PIL-PII-Entities	string	Comma-separated entity types found (e.g. EMAIL,API_KEY)
X-PIL-Cache-Hit	boolean	Whether response was served from semantic cache
X-PIL-Latency-Ms	integer	PIL pipeline latency in milliseconds

2. ML Pipeline — Detailed Design

2.1 Intent Classifier

A distilBERT model fine-tuned on a curated dataset of 500,000 developer prompt pairs labelled across 6 task types and 20 programming languages. Exported to ONNX format for CPU inference via the ONNX Runtime library. Target inference latency: < 20ms p95 on 2 vCPU.

Training Data

- Public: GitHub Copilot community prompt datasets (permissive licence), Stack Overflow accepted answers, HuggingFace code datasets
- Synthetic: 200,000 pairs generated using Claude Sonnet with chain-of-thought labelling
- Label schema: task_type (6 classes), language (20 classes), framework (40 classes), output_format (4 classes)
- Train/val/test split: 80/10/10. Stratified by task_type.

Model Spec

Property	Value
Base model	distilbert-base-uncased
Fine-tune method	Multi-label classification head, 4 output heads
Training compute	8× A100 40GB, ~6 hours
Model size (ONNX)	67MB (quantised INT8: 17MB)
Inference backend	ONNX Runtime 1.17, CPU ExecutionProvider
Latency p50 / p95	8ms / 18ms (2 vCPU c5.large)
Accuracy (task_type)	94.2% on hold-out test set

2.2 Token Budget Engine

The budget engine assigns an information density score to every clause (sentence or code block) in the input prompt, then prunes the lowest-scoring clauses until the prompt fits within the target token budget. The scoring function combines TF-IDF relevance to the classified intent with a structural weight that penalises filler phrases.

Filler phrase examples (always scored 0.0)

"basically", "you know", "I was thinking", "kind of", "like", "in other words"

"Please help me with", "Can you please", "I would appreciate"

Duplicate context: same fact stated more than once receives 0.0 on subsequent occurrences

Vacuous preamble: "I am working on a project that...", "So I have this code and..."

Budget modes:

Mode	Behaviour	When to Use
Strict	Hard cutoff at budget. Lowest-scoring clauses pruned until budget met.	Cost-sensitive pipelines. CI/CD automated calls.
Adaptive	Targets budget but preserves context if quality score would drop > 15%.	Default for interactive developer use.
Economy	Maximum compression. Extracts intent and constraints only.	High-volume batch jobs. Throwaway queries.
Lossless	No compression. PII scrubbing + restructuring only.	Sensitive content. Legal/medical prompts.

2.3 PII Scrubber

Built on Microsoft Presidio with custom extensions for developer-specific entity types not covered by Presidio's default recognisers.

Standard Presidio Entities Enabled

- PERSON, EMAIL_ADDRESS, PHONE_NUMBER, CREDIT_CARD, IBAN_CODE
- US_SSN, US_DRIVER_LICENSE, US_PASSPORT, UK_NHS, IN_AADHAAR
- IP_ADDRESS, URL, MEDICAL_LICENSE, DATE_TIME, LOCATION

Custom PIL Recognisers

Entity Type	Pattern Description	Example
OPENAI_API_KEY	sk- prefix, 48 alphanumeric	sk-aBcDeFgH...
ANTHROPIC_API_KEY	sk-ant- prefix, variable length	sk-ant-api03-...
AWS_ACCESS_KEY	AKIA prefix, 20 uppercase alphanumeric	AKIAIOSFODNN7EXAMPLE
AWS_SECRET_KEY	40-char base64, adjacent to ACCESS_KEY	wJalrXUtnFEMI/K7MDENG
GITHUB_TOKEN	ghp_ or gho_ or github_pat_ prefix	ghp_aBcDe...
PRIVATE_KEY_BLOCK	PEM block header/footer	-----BEGIN RSA PRIVATE KEY-----
DATABASE_CONNECTION	Connection string with credentials	postgresql://user:pass@host
JWT_TOKEN	Three base64 segments separated by dots	eyJhbGciOi...

2.4 Semantic Cache

Prompts are embedded using all-MiniLM-L6-v2 (384 dimensions) and stored in Qdrant. On each new request, the optimised prompt embedding is queried against the cache. If cosine similarity exceeds the threshold, the cached response is returned without an LLM call.

Parameter	Value	Notes
Embedding model	all-MiniLM-L6-v2	CPU-friendly, 384-dim, strong semantic quality
Cache similarity threshold	0.92	Tuned to avoid false cache hits on subtly different prompts
Cache TTL	24 hours	Configurable per customer. Enterprise: 0 (no caching).
Qdrant collection	One per provider + model combination	Prevents cross-provider cache pollution
Index type	HNSW (ef_construct=200, m=16)	Sub-millisecond approximate nearest neighbour
Expected cache hit rate	15–35% in 30 days	Depends on prompt diversity of customer workload

3. Infrastructure Design

3.1 Deployment Architecture

PIL is designed to deploy as a cloud-agnostic Kubernetes workload. The Docker image is < 2GB and contains all ML models baked in — no model download on startup. Cold start time < 30 seconds.

Kubernetes Resource Allocation (per instance)

- Proxy service: 2 vCPU, 2GB RAM — 2 replicas minimum
- ML inference service: 4 vCPU, 8GB RAM — 2 replicas minimum (models loaded in memory)
- Qdrant: 2 vCPU, 4GB RAM, 20GB SSD — 1 instance (single-node for MVP, cluster in V1 GA)
- Redis: 1 vCPU, 2GB RAM — ElastiCache t3.small or equivalent
- Celery workers: 2 vCPU, 2GB RAM — 2 replicas

3.2 Horizontal Scaling

- Proxy service and Celery workers are stateless — scale horizontally on CPU > 70% or queue depth > 500
- ML inference service is stateful (models in memory) — scale slowly, health check before traffic
- Qdrant scales vertically in V1; horizontal sharding deferred to V1 GA
- Redis cluster mode for high availability in enterprise tier

3.3 Latency Budget

Stage	Target p50	Target p95	Optimisation Lever
Auth + rate limit	< 2ms	< 5ms	In-memory token validation
PII scrubber	< 10ms	< 20ms	Presidio async, pre-compiled regex
Semantic cache lookup	< 3ms	< 8ms	Qdrant HNSW, colocated with proxy
Intent classifier	< 10ms	< 20ms	ONNX Runtime, INT8 quantisation
Token budget engine	< 5ms	< 15ms	Pure Python, pre-tokenised
Prompt builder	< 2ms	< 5ms	Template pre-compiled
Total PIL overhead	< 32ms	< 73ms	Well within < 80ms p95 target

3.4 Zero-Retention Mode

When a customer enables zero-retention mode (default for all free-tier and opt-in for cloud-tier), the following guarantees apply at the infrastructure level:

- Prompt content is processed entirely in the FastAPI request handler memory scope
- No prompt content is written to any log file, database, object store, or queue payload
- Audit log entries contain only: timestamp, token_count_in, token_count_out, entity_types_found (not values), provider, model, latency_ms, cache_hit
- Redis payloads contain only the Qdrant vector ID (not the original prompt) for cache invalidation
- ML model inputs are discarded after inference returns

- Celery worker receives only the audit log metadata struct — no prompt content

4. API Reference — Core Endpoints

4.1 Proxy Endpoints

```
POST /openai/v1/chat/completions → proxies to OpenAI
POST /anthropic/v1/messages      → proxies to Anthropic
POST /gemini/v1/generateContent → proxies to Google Gemini
POST /v1/messages                → generic endpoint (provider in header)
```

4.2 Management Endpoints

```
GET /health      → liveness check
GET /ready       → readiness check (models loaded)
GET /stats       → aggregate savings for authenticated user
GET /stats/requests → paginated request history (metadata only)
POST /config/budget → set default budget mode and token limit
POST /config/retention → toggle zero-retention mode
```

4.3 Python SDK Example

```
from pil_sdk import PILClient

client = PILClient(api_key='pil-...', budget_mode='adaptive')

# Drop-in replacement for openai.OpenAI()
response = client.openai.chat.completions.create(
    model='gpt-4o',
    messages=[{'role': 'user', 'content': 'Fix my auth bug...'}]
)

print(response.pil.tokens_saved)      # 847
print(response.pil.compression_ratio) # 0.41
print(response.pil.intent)            # 'debug:python:fastapi'
```

5. Security Design

5.1 Authentication

- PIL API keys: pil-{uuid4} format, stored as bcrypt hashes, never in plaintext
- Per-key rate limits: configurable, default 1000 req/hour
- Key rotation: customers can rotate at any time; old key has 24-hour grace period
- Provider API keys: stored in customer's own environment variables, never transmitted to PIL servers

5.2 Transport Security

- TLS 1.3 minimum on all endpoints
- HTTP Strict Transport Security (HSTS) with 1-year max-age
- No mixed content — all assets served over HTTPS
- Certificate pinning recommended for enterprise SDK deployments

5.3 Input Validation

- Maximum prompt size: 128,000 characters (hard limit)
- Content-Type enforced: application/json only
- JSON schema validation on all request bodies before processing
- Injection protection: prompt content treated as opaque string, never interpolated into system code

Document Classification: Engineering Confidential — Internal Use Only
PIL Engineering Team | Architecture Report v1.0 | March 2026